

Letters and Digits in C

- **Letters:**

- Uppercase letters: A ... Z
- Lowercase letters: a ... z

- **Digits:**

- All decimal digits: 0 ... 9

Special Characters in C

,	comma	&	ampersand
.	period	^	caret
;	semicolon	*	asterisk
:	colon	-	minus sign
?	question mark	+	plus sign
'	apostrophe	<	less than / opening angle bracket
"	quotation mark	>	greater than / closing angle bracket
!	exclamation mark	(	left parenthesis
	vertical bar	)	right parenthesis
/	slash	[	left bracket
\	backslash	]	right bracket
~	tilde	{	left brace
_	underscore	}	right brace
\$	dollar sign	#	number sign
%	percent sign		

White Space Characters in C

White space characters are invisible characters that affect spacing or layout in output:

- (Space): Regular space between tokens.

Example: `int x = 5;`

- `\t` (Horizontal Tab): Inserts a tab space.

Example: `printf("a\tb");`

Output: a b

- `\n` (New Line): Moves cursor to the next line.

Example: `printf("Hello\nWorld");`

Output:

Hello

World

- `\r` (Carriage Return): Returns cursor to line start, overwriting existing text.

Example: `printf("abc\rXYZ");`

Output: XYZ

C Tokens

Definition: In a C program, the smallest individual units are called **tokens**. These are like the words and punctuation in a sentence. C has **six types of tokens** used to construct statements.

Types of C Tokens:

- **Keywords:** `float, while`
- **Constants:** `-15.5, 100`
- **Strings:** `"ABC", "year"`
- **Operators:** `+, -, *, ,`
- **Identifiers:** `main, amount`
- **Special Symbols:** `[ ], { }`

Tokens define the structure and meaning of every valid C statement.

Keywords and Identifiers in C

1. Keywords:

- Keywords are **reserved words** in C that have predefined meanings.
- Cannot be used as names for variables, functions, or identifiers.
- Examples:
 - `int, return, while, float, if`

2. Identifiers:

- Identifiers are **user-defined names** for variables, functions, arrays, etc.
- Examples:
 - `total, main, student_name, _sum`

What is a Variable?

Definition: A **variable** is a named storage location in memory used to hold a value that can be changed during program execution.

Key Points:

- Every variable has a **name**, a **type**, and a **value**.
- The type determines what kind of data it can hold (e.g., `int`, `float`, `char`).
- Variables must be declared before use.

Example:

```
1 int age = 25;  
2 float salary = 45000.50;  
3 char grade = 'A';
```

Rules for Naming Variables in C

Variable names must follow these rules:

1

2

Example: `sum1` is valid; `1sum` is not.

3

Example: `Rate`, `rate`, `RATE` are all different.

4

Example: `int`, `while`, `return` are invalid.

5

`.`, `#`, `$`, `@`, Only letters, digits, and underscore are allowed.

6

7

for variables and functions (as some are reserved for system use).

Tip: Use meaningful names for better readability!

Examples of Variable Names

Variable Name	Valid?	Remark
First_tag	Valid	Follows all rules
char	Not valid	char is a keyword
Price\$	Not valid	Dollar sign is not allowed
group one	Not valid	Contains a blank space
average_number	Valid	Valid, but only first 8 characters are significant in some compilers
int_type	Valid	Keyword may be part of an identifier
3rdvalue	Not valid	Cannot begin with a digit
void	Not valid	void is a keyword
break	Not valid	break is a keyword
total%	Not valid	Percent sign is not allowed
_temp	Valid	Underscore allowed as first character
return	Not valid	Reserved keyword

Data Types in C

Data types define the type of data a variable can store. In C, data types are broadly classified into:

1. Basic Data Types:

- `int` – Integer values *Example:* `int age = 25;`
- `float` – Single precision floating point *Example:* `float height = 5.9;`
- `double` – Double precision floating point *Example:* `double pi = 3.14159;`
- `char` – Single character *Example:* `char grade = 'A';`

2. Derived and User-defined Types:

- Arrays, Structures, Unions, Pointers, Enums
- *Example:* `int scores[5];` (Array of integers)

C Program: Area and Perimeter of a Circle

```
1 #include <stdio.h>
2 #define PI 3.1416
3
4 int main() {
5     float radius, area, perimeter;
6
7     radius=15; //change it as needed
8
9     // Calculate area and perimeter
10    area = PI * radius * radius;
11    perimeter = 2 * PI * radius;
12
13    // Display results
14    printf("Area = %.2f\n", area);
15    printf("Perimeter = %.2f\n", perimeter);
16
17    return 0;
18 }
```

Do It Yourself

Problem: Area and Perimeter of a Rectangle

Write a C program to:

- Input the **length** and **width** of a rectangle.
- Calculate the **area** using the formula: $\text{area} = \text{length} \times \text{width}$
- Calculate the **perimeter** using the formula: $\text{perimeter} = 2 \times (\text{length} + \text{width})$
- Display the results.

Expected Input/Output:

- *Input:* Length = 10, Width = 5
- *Output:* Area = 50, Perimeter = 30

Tip: Use `float` or `int` based on whether decimals are needed.

Temperature Conversion in C

```
1 #include <stdio.h>
2
3 int main() {
4     float celsius, fahrenheit;
5
6     // Celsius to Fahrenheit
7     celsius = 33;
8     fahrenheit = (celsius * 9 / 5) + 32;
9     printf("Temperature in Fahrenheit = %f\n", fahrenheit);
10
11    // Fahrenheit to Celsius
12    fahrenheit = 190;
13    celsius = (fahrenheit - 32) * 5 / 9;
14    printf("Temperature in Celsius = %f\n", celsius);
15
16    return 0;
17 }
```

Program Exercise: Sum Assigned to Float

Problem: Write a program to:

- Declare `x` and `y` as `int` variables, and `z` as a `float` variable.
- Assign two 6-digit numbers to `x` and `y`.
- Assign the sum of `x` and `y` to `z`.
- Output the values of `x`, `y`, and `z`.

Program:

```
1 #include <stdio.h>
2
3 int main() {
4     int x = 123456, y = 654321;
5     float z;
6
7     z = x + y;
8
9     printf("x = %d\\n", x);
10    printf("y = %d\\n", y);
11    printf("z = %f\\n", z); // Sum as float
12
13    return 0;
14 }
```